Solution: Split a String Into the Max Number of Unique Substrings

Problem Statement

Given a string `s`, return the maximum number of unique substrings that the given string can be split into.

You can split string `s` into any list of **non-empty substrings**, where the concatenation of the substrings forms the original string. However, you must split the substrings such that all of them are **unique**.

A substring is a contiguous sequence of characters within a string.

Example 1:

```
Input: s = "aab"
Output: 2
Explanation: Two possible ways to split the given string into maximum
unique substrings are: ['a', 'ab'] & ['aa', 'b'], both have 2
substrings; hence the maximum number of unique substrings in which the
given string can be split is 2.
```

Example 2:

```
Input: s = "abcabc"
Output: 4
Explanation: Four possible ways to split into maximum unique substrings
are: ['a', 'b', 'c', 'abc'] & ['a', 'b', 'cab', 'c'] &  ['a', 'bca',
'b', 'c'] & ['abc', 'a', 'b', 'c'], all have 4 substrings.
```

Solution

We can use backtracking to solve this problem.

This solution uses a helper function `splitAndCount` which takes three arguments, the input string `s`, the current start position and a set `set` to keep track of the unique substrings that have been split so far. The `maxUniqueSplit` function calls the `splitAndCount` function to find the maximum number of unique substrings that the given string can be split into.

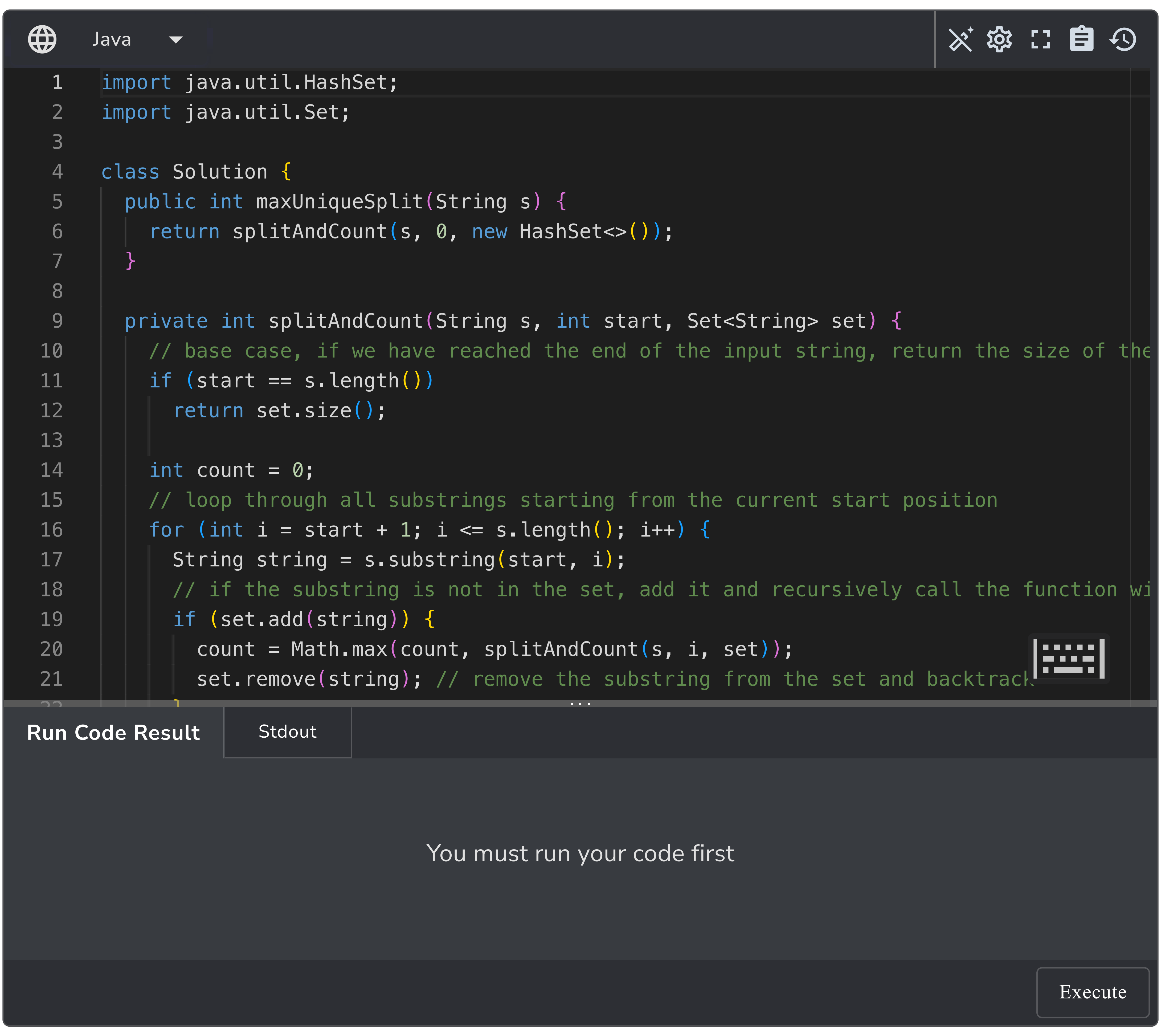
The `splitAndCount` function starts with a base case where it returns the size of the set when the current start position is equal to the length of the input string. This means that all substrings have been processed and the size of the set represents the maximum number of unique substrings.

The function then uses a for loop to iterate through all possible substrings starting from the current start position. For each substring, it checks if the substring is already in the set. If it is not, the substring is added to the set and the function is recursively called with the new start position being the end of the current substring. This continues until all possible substrings have been processed.

After the recursive call, the substring is removed from the set to backtrack. The function keeps track of the maximum number of unique substrings found so far and returns this maximum count when all substrings have been processed.

Code

Here is the code of our algorithm:



```java
import java.util.HashSet;
import java.util.Set;

class Solution {
  public int maxUniqueSplit(String s) {
    return splitAndCount(s, 0, new HashSet<>());
  }

  private int splitAndCount(String s, int start, Set<String> set) {
    // base case, if we have reached the end of the input string, return the size of the
    if (start == s.length())
      return set.size();

    int count = 0;
    // loop through all substrings starting from the current start position
    for (int i = start + 1; i <= s.length(); i++) {
      String string = s.substring(start, i);
      // if the substring is not in the set, add it and recursively call the function wi
      if (set.add(string)) {
        count = Math.max(count, splitAndCount(s, i, set));
        set.remove(string); // remove the substring from the set and backtrack
```

You must run your code first

Time Complexity

We can split any given string of length `n` in 2^n ways. Hence the time complexity will be $O(2^n)$.

Space Complexity

The space complexity will be $O(n)$ as we need to save only one way of splitting the given string while in the recursion, and our recursion tree won't get bigger than $O(n)$ steps too.

← Previous
Split A String Into The Max Number Of Unique Substrings (Medium)

Next →
Introduction To Trie

☐ Mark as Completed